MTH/CSC 4150
Fall 2025
Draft Project Report:
Mathematically Modeling a Distance Runner's Race with Interpolation

Daron Baltazar, Evan Dant, Madi Price

Friday, December 5th

1 Introduction

Researchers have developed new ways to predict running performance and maximize potential (Dash, 2024). Different forms of regression are common models that they use (Millett & Melanson, n.d.), but in using numerical methods, interpolation will also provide valuable insight into runner performance. Efforts have already been made to calculate the time it is expected a runner to take in comparison to the distance of a race using both interpolation and extrapolation, as displayed in this web link.

We model the cumulative distance of the runner as a function of time denoted by $s(t)$. From this interpolated position function, we approximate the instantaneous velocity $v(t)$ and the acceleration $a(t)$. We compare three interpolation approaches: a Newton interpolating polynomial, a natural cubic spline, and a clamped cubic spline. We then examine how well each method captures realistic pacing patterns, and specifically how spline boundary conditions influence estimates of velocity and acceleration.

2 Theoretical Background

2.1 Polynomial Interpolation

Suppose there are $n+1$ data points (t_i, s_i) with distinct time values $t_0 < t_1 < \cdots < t_n$. An interpolating polynomial $p_n(t)$ of degree at most n satisfies

$$p_n(t_i) = s_i, \qquad i = 0, 1, \ldots, n. \tag{1}$$

There exists a unique polynomial of degree at most n that satisfies (1).

In the Lagrange form,

$$p_n(t) = \sum_{i=0}^{n} s_i L_i(t), \tag{2}$$

where the Lagrange basis polynomials are

$$L_i(t) = \prod_{\substack{j=0 \\ j \neq i}}^{n} \frac{t - t_j}{t_i - t_j}, \qquad i = 0, 1, \ldots, n. \tag{3}$$

This basis is interpolatory, since $L_i(t_j) = \delta_{ij}$.

In Newton's form, an alternative basis is used:

$$p_n(t) = a_0 + a_1(t - t_0) + a_2(t - t_0)(t - t_1) + \cdots + a_n \prod_{k=0}^{n-1}(t - t_k), \tag{4}$$

where the coefficients a_i are computed from divided differences. Newton's form is more convenient for computation and for updating the polynomial when additional data points are added.

2.2 Interpolation Error

Let $f(t)$ be a sufficiently smooth function and let $p_n(t)$ be the degree-n interpolating polynomial constructed from $n+1$ distinct nodes $t_0, \ldots, t_n$. The interpolation error can be written as

$$f(t) - p_n(t) = \frac{f^{(n+1)}(c)}{(n+1)!} \prod_{i=0}^{n}(t - t_i), \tag{5}$$

for some c in the interval containing the nodes and the point t. This formula shows that the error depends on both the spacing of the nodes and the $(n+1)$st derivative of f. When the nodes are equally spaced and n is large, the product term can lead to large oscillations near the ends of the interval (Runge's phenomenon).

2.3 Cubic Splines

To reduce the oscillations associated with high-degree polynomials, we use cubic splines. Given $n+1$ knots $t_0 < t_1 < \cdots < t_n$ and function values $s_i = f(t_i)$, a cubic spline $S(t)$ of the form

$$S(t) = \begin{cases} S_0(t), & t \in [t_0, t_1], \\ S_1(t), & t \in [t_1, t_2], \\ \vdots & \\ S_{n-1}(t), & t \in [t_{n-1}, t_n], \end{cases} \tag{6}$$

where each $S_i(t)$ is a cubic polynomial

$$S_i(t) = a_i + b_i(t - t_i) + c_i(t - t_i)^2 + d_i(t - t_i)^3. \tag{7}$$

The spline is constructed so that

1. $S(t_i) = s_i$ and $S(t_{i+1}) = s_{i+1}$ for $i = 0, \ldots, n$ (interpolation),
2. $S(t)$ is continuously differentiable, so $S'(t)$ is continuous at each interior knot $t_1, \ldots, t_{n-1}$,
3. $S''(t)$ is continuous at each interior knot.

These conditions yield $4n - 2$ equations. Two additional conditions are needed at the endpoints. In this project, we use the *natural cubic spline*, which imposes

$$S''(t_0) = 0, \qquad S''(t_n) = 0. \tag{8}$$

Alternatively, a *clamped cubic spline* specifies the endpoint slopes

$$S'(t_0) = v_0, \qquad S'(t_n) = v_n, \tag{9}$$

for prescribed velocities v_0 and v_n. This allows the model to encode physically meaningful information about the runner's starting and ending pace.

Solving the resulting linear systems produces the coefficients a_i, b_i, c_i, d_i for all subintervals.

2.4 Numerical Differentiation

Once a smooth interpolating function $S(t)$ is available, the velocity and acceleration of the runner are given by

$$v(t) = S'(t), \qquad a(t) = S''(t). \tag{10}$$

For spline models, derivatives can be computed analytically by differentiating the cubic expressions, or numerically using finite difference stencils.

For comparison, numerical differentiation formulas can also be derived directly from discrete data, or from a given interpolant evaluated on a fine time grid. For example, the centered difference approximation for the first derivative at an interior node is

$$f'(t_i) \approx \frac{f(t_{i+1}) - f(t_{i-1})}{2h}, \tag{11}$$

where $h = t_{i+1} - t_i$ is the time step. This stencil is second order accurate in h.

3 Data and Preprocessing

The data for this project consist of split times and cumulative distances from a single distance runner's race. Let t_i denote the elapsed time at the ith checkpoint and let s_i denote the cumulative distance at this time.

We use data from the New York City Marathon, with splits every 5 kilometers. The first runner we chose to examine is Fiona O'Keeffe, the first female finisher for the USA. We were also given data from another, more average, female runner. Both of the splits are displayed in Figure **??**.

O'Keeffe Split Times

Figure 1: NYC Marathon split times for Fiona O'Keeffe.

We perform the following preprocessing steps:

1. Convert all distances to meters and all times to seconds.
2. Shift the time axis so that $t_0 = 0$ at the race start.
3. Remove any obvious measurement errors such as negative time differences between consecutive splits.

The resulting data $\{(t_i, s_i)\}_{i=0}^{n}$ serves as the input to all interpolation methods.

4 Numerical Methods and Implementation

4.1 Interpolation Models

We implement and compare the following models.

1. **Polynomial interpolation.** We construct the Lagrange and Newton interpolating polynomials using custom MATLAB functions based on (**??**)–(**??**). This approach provides a baseline but is expected to be sensitive to node spacing and prone to oscillations.
2. **Natural cubic spline.** We use MATLAB's `spline` command to construct the natural cubic spline that satisfies (**??**). We also write helper functions that evaluate $S(t)$, $S'(t)$, and $S''(t)$ analytically using the spline coefficients.
3. **Clamped cubic spline.** We construct a clamped spline that satisfies the endpoint slope conditions in (**??**), where v_0 and v_n are estimated using one-sided finite differences at the start and end of the race. This allows us to encode a near-zero starting velocity and an empirically-based ending velocity consistent with observed pacing.

4.2 Velocity and Acceleration

For the spline models, we compute the velocity and acceleration by differentiating the spline pieces analytically. For a cubic

$$S_i(t) = a_i + b_i(t - t_i) + c_i(t - t_i)^2 + d_i(t - t_i)^3,$$

the derivatives are

$$S_i'(t) = b_i + 2c_i(t - t_i) + 3d_i(t - t_i)^2, \tag{12}$$

$$S_i''(t) = 2c_i + 6d_i(t - t_i). \tag{13}$$

Evaluating (**??**) and (**??**) yields approximate velocity and acceleration profiles for the runner.

For the polynomial model, we evaluate $p_n(t)$ on a fine, equally spaced time grid and apply the centered-difference stencil (**??**) to obtain $v(t)$, followed by a second application of the stencil to approximate $a(t)$. This provides a consistent finite-difference framework for comparing polynomial- and spline-based derivatives.

4.3 Accuracy Checks

We validate the interpolants in one main way.

1. **Checkpoint consistency.** For each method, we verify that the interpolant reproduces the input distances at the original split times, that is $S(t_i) \approx s_i$.

5 Results

5.1 Position vs. Time

Figure **??** displays cumulative distance versus elapsed time for the Newton interpolating polynomial, the natural cubic spline, the clamped cubic spline, and the original split data. All three interpolation methods pass exactly through each data point, as required by construction, and they are nearly indistinguishable over the majority of the race.

Small differences among the interpolants appear primarily near the endpoints of the interval. The Newton polynomial shows a slightly sharper curvature near the final split compared to both spline models, which is consistent with the oscillatory behavior associated with high-degree polynomial interpolation near boundaries. The natural and clamped splines remain visually smoother at the ends of the race, with the clamped spline exhibiting the most uniform curvature due to its imposed endpoint slope conditions.

Because the underlying split data are approximately linear for this elite runner, the position plots alone do not differentiate the interpolants strongly. The influence of the interpolation choice becomes more pronounced after differentiation, as seen in the velocity and acceleration results below.

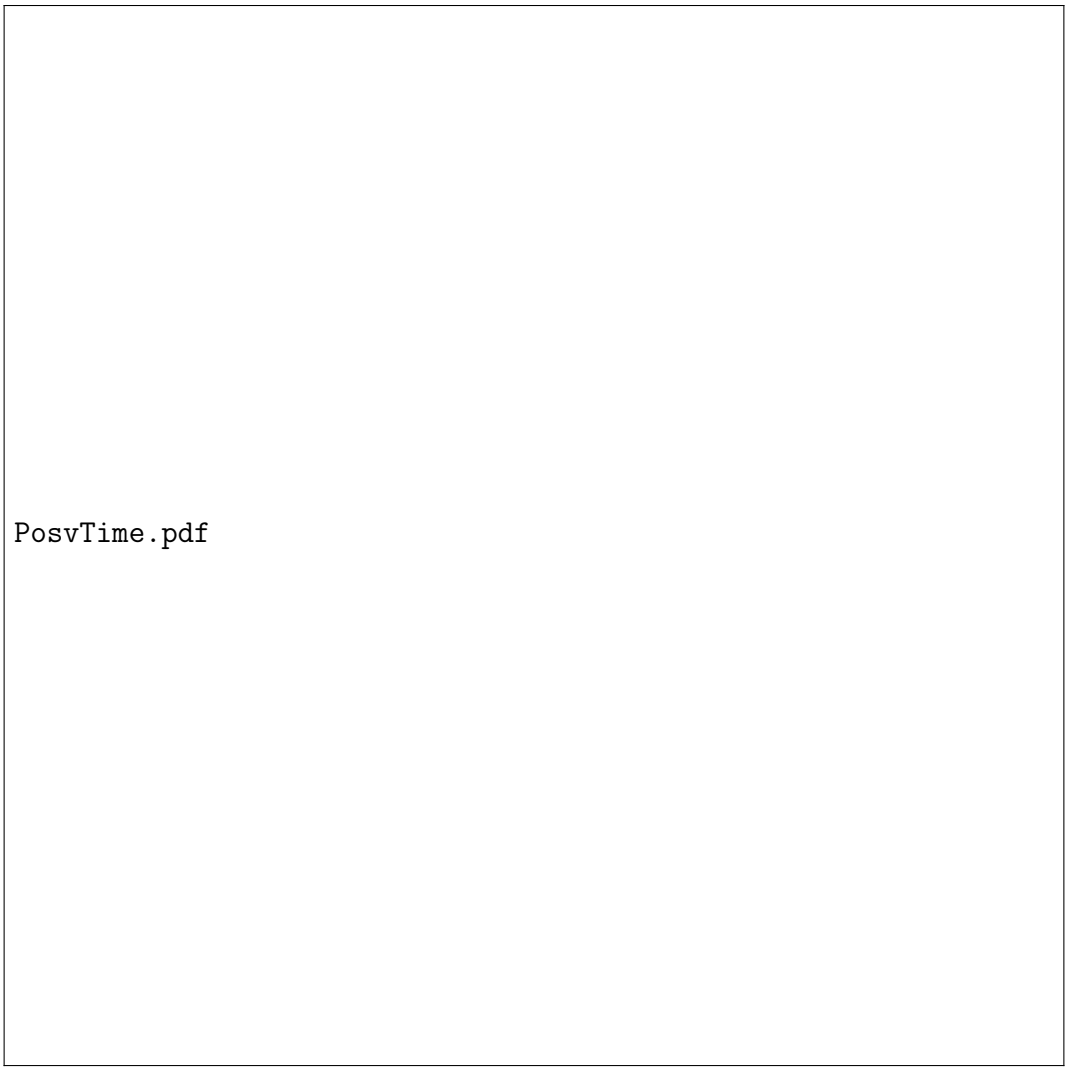

Figure 2: Cumulative distance versus time for the Newton polynomial, natural spline, and clamped spline, together with the original split data.

5.2 Velocity Profiles

Figure **??** displays velocity versus time computed from each interpolation method. The velocity associated with the Newton polynomial was computed using centered finite-difference approximations applied to the polynomial interpolant, while the spline velocities were computed analytically from the derivatives of the cubic piecewise functions.

At the beginning of the race, the natural cubic spline shows a nonzero initial velocity. This behavior is a direct consequence of the natural spline's boundary condition, $S''(t_0) = 0$, which suppresses acceleration at the start of the interval and prevents the model from capturing the sharp initial acceleration from rest. In contrast, the polynomial-based velocity and the clamped spline velocity both begin near zero, which is more consistent with physical race dynamics.

Throughout the middle portion of the race, all three models show similar velocity trends

with minor fluctuations corresponding to small pacing adjustments. Near the final split, the most notable difference occurs: both spline models—especially the clamped spline—display a modest increase in velocity consistent with an end-of-race surge commonly observed in competitive runners. The Newton polynomial, however, predicts a decline in velocity during this phase, an artifact of boundary oscillation and global polynomial behavior rather than genuine pacing characteristics reflected in the original data.

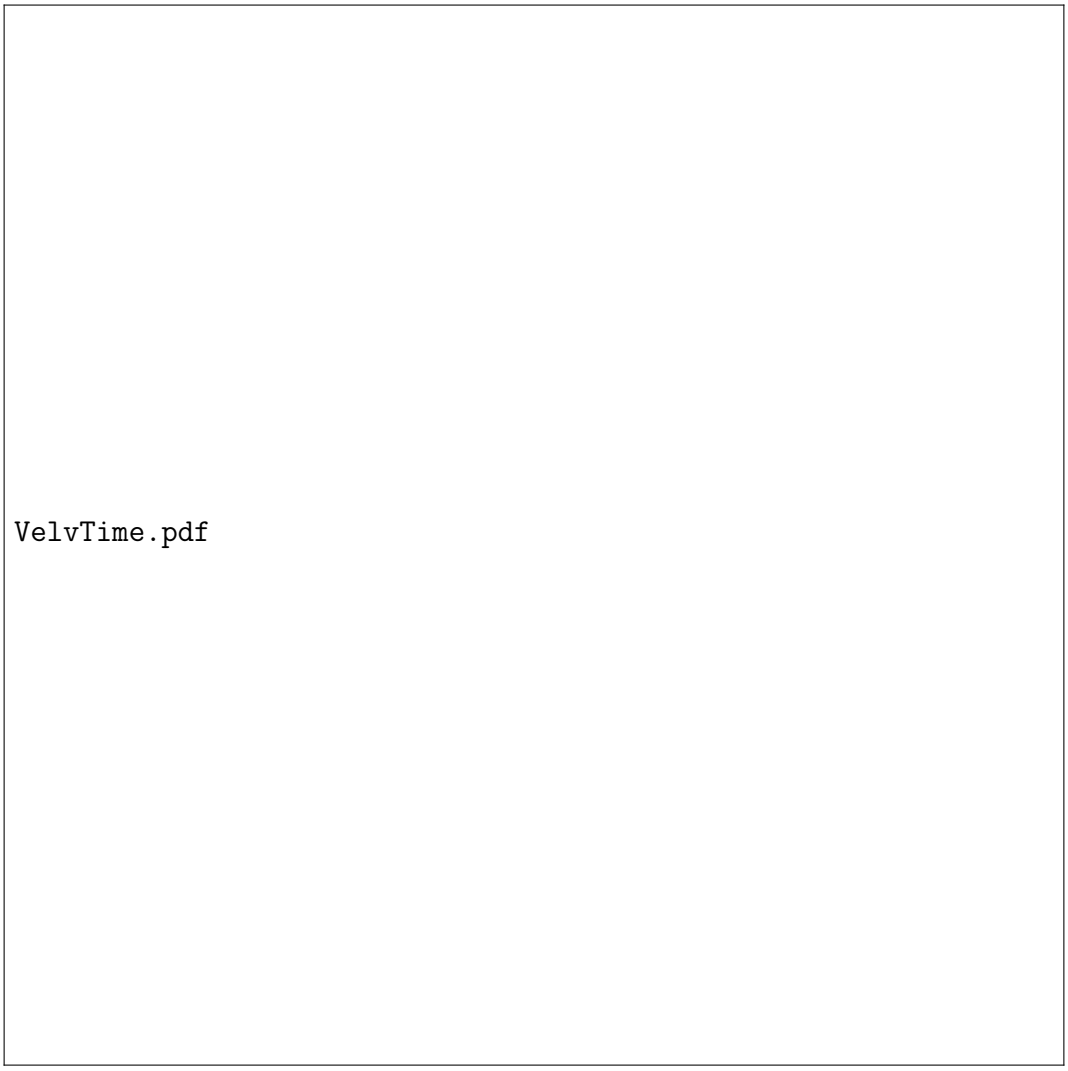

Figure 3: Velocity versus time obtained from the polynomial and spline interpolations.

5.3 Acceleration Profiles

Figure **??** displays acceleration versus time for each interpolant. The polynomial acceleration was computed using finite-difference differentiation of the polynomial-derived velocities, while spline accelerations were obtained analytically from the second derivatives of the cubic splines.

The natural spline's acceleration curve remains near zero at both endpoints, reflecting

its enforced boundary conditions $S''(t_0) = S''(t_n) = 0$. While mathematically smooth, this behavior does not accurately represent the physical start of the race, where substantial positive acceleration is expected as the runner departs from rest.

The polynomial interpolation predicts a large positive acceleration at the beginning of the race followed by a decreasing trend and subsequent oscillations later in the race. Although the large initial acceleration is physically reasonable, the oscillatory swings thereafter are not supported by observable pace changes in the data.

The clamped spline strikes a balance between these two extremes. By enforcing physically meaningful endpoint velocity conditions rather than zero curvature constraints, the clamped model allows nonzero accelerations near the boundaries while maintaining the smooth interior behavior of cubic splines. This produces a more realistic acceleration profile throughout the race.

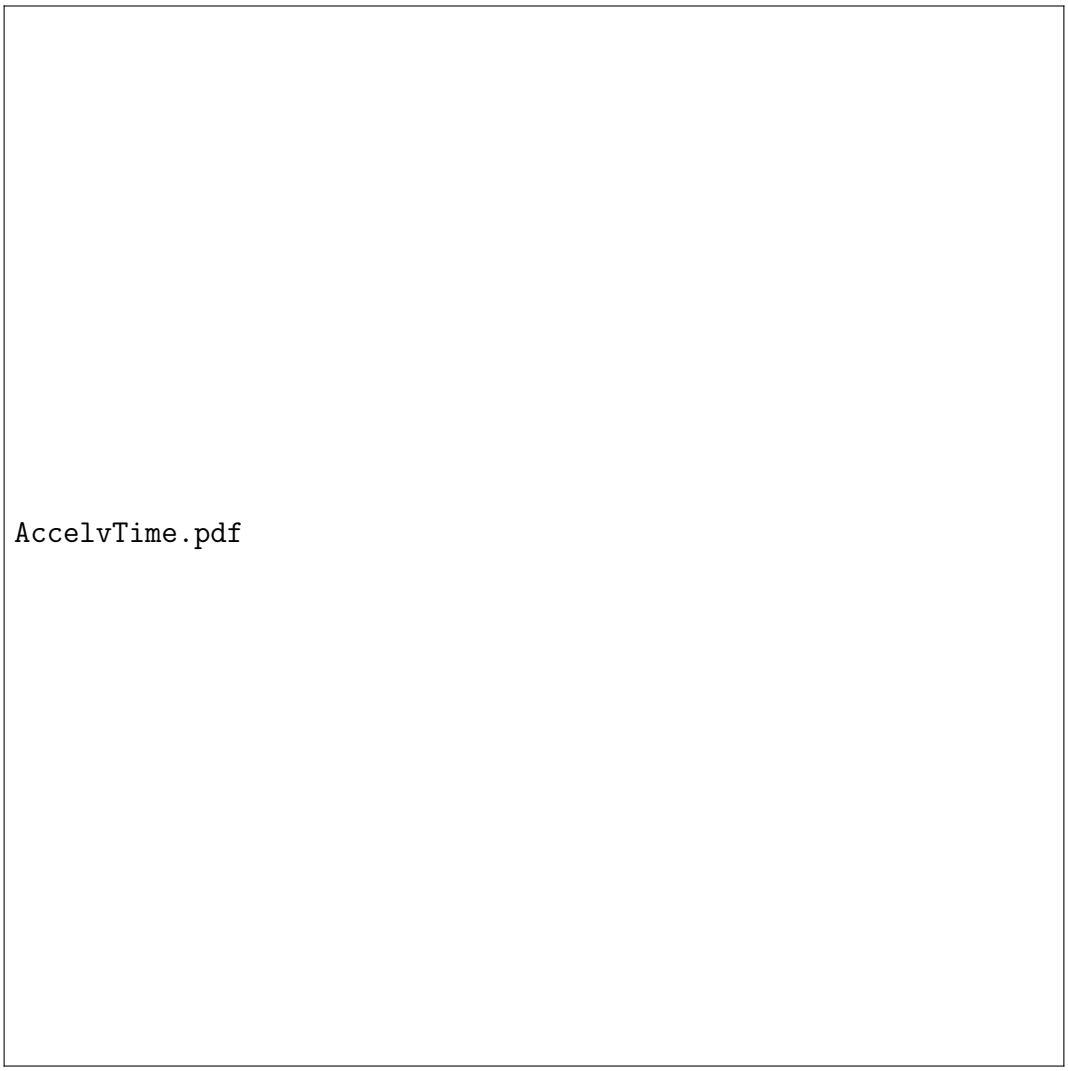

Figure 4: Acceleration versus time from each interpolation method.

6 Discussion

The comparative analysis reveals distinct strengths and limitations across the interpolation methods examined. The Newton polynomial effectively captures sharp changes near the start of the race, including large initial acceleration and a velocity curve beginning near zero. However, its susceptibility to boundary oscillations generates less realistic behavior later in the race, particularly the artificial velocity decline near the finish observed in Figure **??**.

The natural cubic spline provides the smoothest overall curves but exhibits artificially constrained endpoint behavior due to the imposed conditions $S''(t_0) = S''(t_n) = 0$. These constraints suppress acceleration at both the start and finish of the race, producing a nonzero starting velocity and nearly zero terminal acceleration in Figure **??**. While mathematically attractive, these features conflict with the physical dynamics of distance running, specifically the acceleration from rest at the race start and the surge often observed near the finish.

The clamped cubic spline offers the most physically plausible model across all stages of the race. By prescribing realistic endpoint velocities instead of zero curvature conditions, the clamped spline preserves the desirable smoothness of cubic splines while allowing accurate representation of early acceleration and late-race surging behavior. Its velocity and acceleration profiles align more consistently with race dynamics than either the natural spline or the polynomial interpolant.

Overall, while all methods reconstruct the position data accurately, the differentiation process magnifies modeling assumptions. The results demonstrate that careful selection of spline boundary conditions is essential when derivative quantities such as velocity and acceleration are of primary interest. Among the interpolants tested, the clamped cubic spline provides the most balanced and physically realistic representation of the runner's pacing profile.

7 Conclusion and Future Work

This project demonstrates that polynomial and spline interpolations are useful tools for modeling a distance runner's race based on discrete split data. Among the methods tested, the clamped cubic spline interpolation produces the most realistic pacing curves for the majority of the race. High-degree polynomials perform well at the beginning of races, but can oscillate and generate potentially unrealistic derivatives towards the middle and ends of the race, while natural splines can impose endpoint behavior that is overly restrictive from a physical standpoint.

Future work could extend the current model by incorporating additional race information, such as elevation changes, or by incorporating numerical integration methods. Analyzing multiple runners across multiple demographics could also yield interesting insights as to the effect that classifiers such as age, gender, and race have on long distance running.

References

1. Dash, S. (2024). Win Your Race Goal: A Generalized Approach to Prediction of Running Performance. *Sports Medicine International Open*, 8, a24016234. https://doi.org/10.1055/a-2401-6234

2. Millett, M., & Melanson, T. (n.d.). Predicting Running Times from Race History.

3. T. Sauer, *Numerical Analysis*, 3rd ed., Pearson, 2019.

4. Course notes and lecture slides from MTH/CSC 4150, Fall 2025.

5. New York Road Runners. (2025, November 2). *2025 TCS New York City Marathon: Finishers results*. `https://results.nyrr.org/event/M2025/result/108?_gl=1*1ptu7wj*_gcl_au*MjAyNjE3NzQzMS4xNzYyMjg2OTg5`